MVCC

row versions, snapshots, xmin/xmax & VACUUM

Watch the 4-min video

In one line: with multi-version concurrency control, every write creates a new row version instead of overwriting, and every reader sees a consistent snapshot. Readers never block writers and writers never block readers; the price is dead versions that VACUUM must clean up.

1 The problem: a long report next to live traffic

The finance team runs a **20-minute analytics query** on your production Postgres, scanning every order. Meanwhile thousands of customers are placing new orders. In an old lock-based database, the report would block the writes, or the writes would block the report. In Postgres, neither waits. That is **MVCC**.

The idea: keep **several versions** of a row instead of one. A write never overwrites a row in place; it creates a new version. Each reader sees whichever version was current for its snapshot. So:

- **Readers never block writers**, and **writers never block readers**.
- Only **two writers changing the same row** still wait for each other (row locks, next lesson).

2 Row versions: xmin and xmax

In Postgres every row version (a **tuple**) carries two hidden fields:

- `xmin`: the ID of the transaction that **created** this version.
- `xmax`: the ID of the transaction that **deleted or replaced** it (0 while it is live).

So an `UPDATE` is really two steps: set the old version's `xmax`, and insert a new version with a new `xmin`. A `DELETE` only sets `xmax`. You can see them yourself: `SELECT xmin, xmax, * FROM orders WHERE id = 42;`

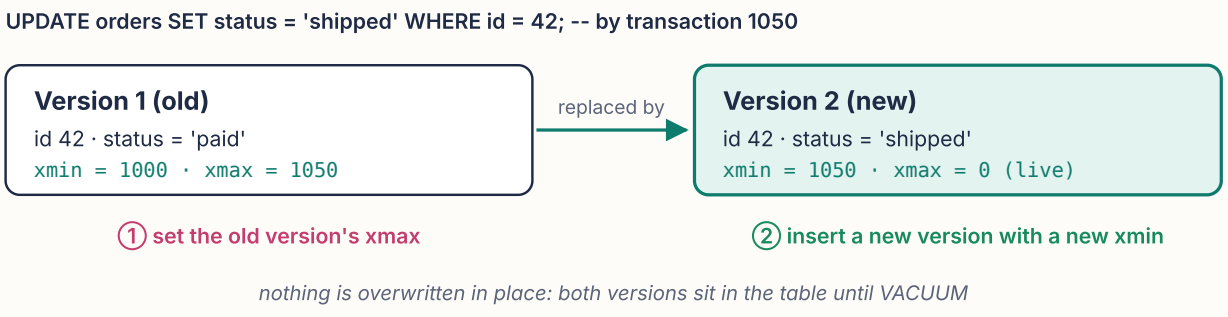


Figure 1. An update by transaction 1050 marks version 1 as replaced and adds version 2. Which one a reader sees depends on its snapshot.

3 Snapshots and the visibility rule

A transaction reads through a **snapshot**: the set of transactions that had committed at the moment it was taken. A row version is visible when:

```
visible = creator (xmin) committed before the snapshot
AND deleter (xmax) is empty, aborted, or not committed in the snapshot
```

So the 20-minute report sees the database **exactly as it was when it began**, perfectly consistent, while new orders keep committing alongside it. (In Postgres, read committed takes a new snapshot for each statement; repeatable read keeps one snapshot for the whole transaction.)

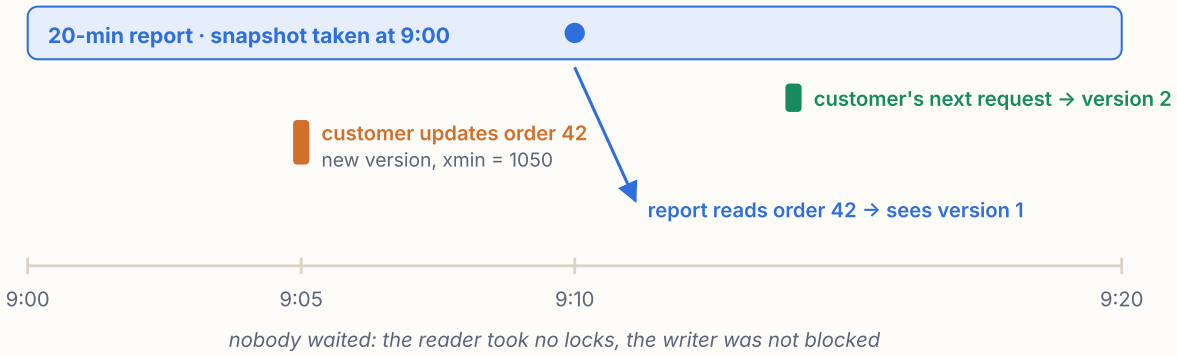


Figure 2. The report's snapshot is from 9:00, so at 9:10 it still reads the old version of order 42. The customer, with a newer snapshot, sees the new one.

WORKED EXAMPLE

- **9:00**: the report starts and takes its snapshot.
- **9:05**: a customer updates order 42. Postgres writes a new version, created by transaction **1050**, and sets the old version's xmax to 1050.
- **9:10**: the report reaches order 42. Transaction 1050 is not in its snapshot, so the new version is invisible and the old one is still visible. It reads the old version.
- The customer's next request takes a fresh snapshot, which includes 1050, and sees the new version. **Nobody waited.**

4 The cost: dead versions and VACUUM

Old row versions that no running transaction can see any more are garbage. They **bloat** tables and indexes and slow down scans. Postgres's **VACUUM**, usually run automatically by **autovacuum**, removes them and makes the space reusable.

The catch: VACUUM can only remove a version once **no snapshot could still need it**. A very long transaction, or a session left **idle in transaction**, holds back cleanup for the whole database, not just its own tables.

MySQL (InnoDB) and Oracle keep old versions in **undo logs** instead of in the table, and rebuild old versions from them on read. The trade-off is similar: long transactions make the undo history grow.

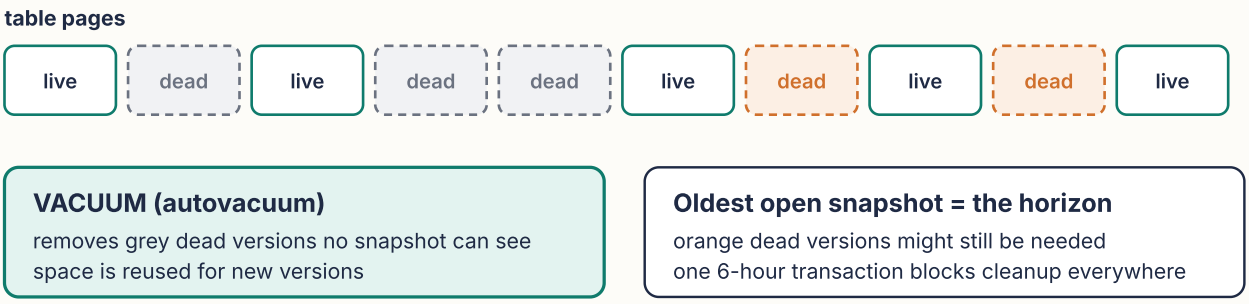


Figure 3. Grey dead versions are invisible to every snapshot and can go. Orange ones were deleted after the oldest open snapshot was taken, so that snapshot may still need them.

5 In practice

- **Keep transactions short.** Never hold one open while waiting for a user or an external API.
- **Set an idle timeout**: `idle_in_transaction_session_timeout` kills sessions that forget to commit.
- **Monitor bloat and autovacuum**: dead tuple counts and last autovacuum time in `pg_stat_user_tables`.
- **Run heavy analytics on a read replica**, not the primary.
- **Avoid pointless updates.** Every update creates a new version, and often new index entries too. Skip writes that change nothing (`WHERE status IS DISTINCT FROM 'shipped'`).

COMMON MISTAKES

- Leaving a transaction open in a console or a connection pool ("idle in transaction") for hours.
- Disabling or starving autovacuum on busy tables, then wondering why the table is 10× its real size.
- Thinking MVCC removes all locking: two writers on the same row still wait.
- Using a column that's updated on every request (a "last seen" timestamp) on a heavily indexed table.

6 Key terms

MVCC: keep many row versions; readers use snapshots

Snapshot: which transactions count as committed for me

Bloat: space wasted by dead versions

Tuple: one physical version of a row

Dead tuple: a version no snapshot can see

Idle in transaction: open transaction doing nothing

xmin / xmax: creating / deleting transaction ID

VACUUM: removes dead tuples, reclaims space

Undo log: where MySQL/Oracle keep old versions

7 Interview questions

> **Q1. How does MVCC let readers and writers avoid blocking each other?** BASIC
> Writes never update rows in place; each creates a new version tagged with the IDs of the transactions that created and deleted it. Each transaction reads from a snapshot and sees only versions committed before it. Readers need no locks, and writers don't wait for readers.

Q2. What are xmin and xmax? BASIC

Hidden Postgres fields on every row version: xmin is the transaction that created it, xmax the one that deleted or replaced it. An UPDATE sets the old version's xmax and inserts a new version with a new xmin.

Q3. What does VACUUM do, and why is it needed? BASIC

It removes dead row versions that no snapshot can see any more and makes their space reusable. Without it, every update leaves garbage behind and tables and indexes grow without limit.

Q4. A table with 1 million live rows takes 40 GB on disk and is slow to scan. What's going on? SCENARIO

Bloat: dead versions far outnumber live ones. Check pg_stat_user_tables for dead tuples and the last autovacuum time, look for long or idle-in-transaction sessions holding the horizon back, and tune autovacuum for that table.

Q5. Someone left a psql session open with BEGIN for 6 hours. Why does that hurt other tables? SCENARIO

Its snapshot might still need old versions anywhere in the database, so VACUUM cannot remove any version deleted after that snapshot was taken. Bloat grows everywhere. Set idle_in_transaction_session_timeout, e.g. 5 minutes.

Q6. Where should the 20-minute finance report run? SCENARIO

On a read replica. On the primary it competes for I/O and its long snapshot holds back cleanup. Note: with hot_standby_feedback on, a long replica query can still hold back vacuum on the primary.

Q7. Do two transactions updating the same row ever wait under MVCC? TRICKY

Yes. MVCC removes reader-writer blocking only. The second writer waits on the first's row lock until it commits or aborts; if the first commits, a repeatable-read writer then fails with a serialization error.

Q8. How do MySQL and Oracle differ from Postgres here? TRICKY

They update the row in place and keep the old versions in undo logs, building older versions from undo when a snapshot needs them. Postgres keeps old versions in the table itself, hence VACUUM. Either way, long transactions make the old-version history grow.

Next: SD-E-011 · Database Locks & Deadlocks backendroadmap.com